

Forwarding, also called **bypassing**, is a crucial hardware technique used to resolve **data hazards** in a pipelined processor, preventing the pipeline from stalling unnecessarily.

1. Explanation of Forwarding

Forwarding works by retrieving the necessary data element early from internal processor buffers (the pipeline registers) rather than waiting for the data to be written into the programmer-visible registers or memory.

The Problem Forwarding Solves: Data hazards occur when an instruction depends on the result of an earlier instruction that is still progressing through the pipeline. For example, if an add instruction is immediately followed by a sub instruction that uses the sum produced by the add:

```
add x19, x0, x1
```

```
sub x2, x19, x3
```

In a typical five-stage RISC-V pipeline (Instruction Fetch (IF), Instruction Decode/Register Read (ID), Execute (EX), Memory Access (MEM), Write Back (WB)), the add instruction does not write its result into the register file until the fifth stage (WB). Without intervention, the sub instruction, which needs the result of add in its third stage (EX), would have to wait, wasting clock cycles.

The Solution: The observation is that the desired result is often calculated and available earlier in the pipeline. For an R-format instruction like add, the result is available at the end of the EX stage. Forwarding adds **multiplexors** to the input of the Arithmetic Logic Unit (ALU). This hardware allows the ALU inputs (the source operands) to be sourced not only from the register file (via the ID/EX pipeline register) but also directly from the outputs of later pipeline registers (EX/MEM or MEM/WB).

By forwarding the result as soon as it is available to the units that need it, the pipeline can often run at full speed, eliminating stalls caused by common data dependencies.

2. Conditions for Detecting and Implementing Forwarding

Forwarding logic must be implemented in the EX stage (where the ALU multiplexors are located) and relies on comparing register numbers stored in the pipeline registers to detect a hazard. The instruction read register numbers (Rs1 and Rs2) are passed from the ID stage to the ID/EX register to facilitate this detection.

The logic checks for two primary hazard types relative to the current instruction's EX stage:

A. EX Hazard (Result available from the immediately preceding instruction)

This condition forwards the result from the instruction currently in the **EX/MEM** pipeline register to the ALU inputs of the instruction currently in the EX stage. This handles instructions that are one cycle ahead.

- **Condition:** The register being written by the instruction in the EX/MEM stage matches a register being read by the instruction in the ID/EX stage.

The conditions for detection are:

1. **If the current instruction needs the result of the instruction one cycle prior (EX/MEM):**

- The instruction in EX/MEM must be writing a register (EX/MEM.RegWrite is asserted).
- The destination register is not register zero (EX/MEM.RegisterRd $\neq$ 0).
- The destination register number in EX/MEM matches the first source register (ID/EX.RegisterRs1). (**ForwardA = 10**)
- OR matches the second source register (ID/EX.RegisterRs2). (**ForwardB = 10**)

B. MEM Hazard (Result available from the instruction two cycles preceding)

This condition forwards the result from the instruction currently in the **MEM/WB** pipeline register to the ALU inputs of the instruction currently in the EX stage. This handles instructions that are two cycles ahead.

- **Condition:** The register being written by the instruction in the MEM/WB stage matches a register being read by the instruction in the ID/EX stage.

The priority rule for forwarding is critical: **The data must be forwarded from the most recent available stage**. Therefore, the forwarding logic must prioritize the EX hazard over the MEM hazard.

The complex conditions for the MEM hazard, ensuring lower priority than the EX hazard, are:

1. If the current instruction needs the result of the instruction two cycles prior (MEM/WB):

- The instruction in MEM/WB must be writing a register (MEM/WB.RegWrite is asserted) and the destination register is not zero (MEM/WB.RegisterRd $\neq$ 0).
- **AND NOT** (The result is already being forwarded from the EX/MEM stage).
- **AND** The destination register number in MEM/WB matches the first source register (ID/EX.RegisterRs1). (**ForwardA = 01**)
- OR matches the second source register (ID/EX.RegisterRs2). (**ForwardB = 01**)

Forwarding works very well, but it **cannot prevent all pipeline stalls**. A stall is still required in the case of a **load-use data hazard**, where an instruction immediately follows a load instruction and tries to use the loaded data. Since the data is not available until after the MEM stage of the load instruction, it is too late for the following instruction's EX stage, requiring a one-cycle stall (or "bubble").

Forwarding in a processor pipeline is like a chef immediately handing a sliced onion to a sous-chef who needs it for the next dish, instead of placing it on the storage shelf first and waiting for the sous-chef to retrieve it later. By "bypassing" the formal storage location (the register file), the kitchen (the pipeline) runs faster.